Python for Cybersecurity

Beginner study notes

Python basics, text and lists, regular expressions, file handling, and debugging

Personal study and revision reference

Contents

Contents 2

Source and coverage 3

What Python adds to security work 3

1. Python building blocks 3

2. Conditional and iterative statements 4

3. Functions and readable code 4

4. Strings and lists 5

5. Regular expressions 5

6. Automation and file handling 6

7. Debugging and common mistakes 6

Security and quality reminders 7

Revision questions 7

References 7

These notes introduce the Python concepts shown in the linked Google Cybersecurity Certificate video and explain them with small, defensive examples. The examples are original study exercises, not copied course code.

Source and coverage

- **Video:** Fundamentals of Python for Cybersecurity | Google Cybersecurity Certificate
- **Creator:** Grow with Google
- **Public chapter markers:** Python introduction and core components; conditionals and loops; functions; readable code; strings and lists; regular expressions; automation; file handling; debugging.
- **Source limitation:** The public video page exposed its description and chapter markers, but a full transcript was not available during this review. The notes follow those documented topics and use original examples; they do not claim to reproduce every explanation or exercise in the video.

What Python adds to security work

Python can automate repetitive, well-defined tasks: checking a list of indicators against a local dataset, summarizing log lines, validating a text file, or preparing data for review. Automation is useful when it makes a repeatable task faster and less error-prone. A script still needs a clear scope, safe inputs, reviewable output, and human verification before consequential actions.

For this guide, examples read only small files that you create yourself. They do not scan external systems, access accounts, or change security settings.

1. Python building blocks

Values, variables, and data types

A **value** is a piece of data. A **variable** is a name referring to a value. Common built-in types include strings (str), integers (int), floating-point numbers (float), and booleans (bool). Use the type that fits the meaning of the data.

```
analyst = "Riya"
failed_attempts = 4
is_authorized = False

print(analyst)
print(failed_attempts + 1)
print(is_authorized)
```

Example output:

```
Riya
5
False
```

An IP address or user name is usually text for a beginner script, even when some characters are digits. Keeping it as a string avoids accidental arithmetic or loss of formatting.

Expressions and operators

An **expression** combines values and operators to produce a value. Comparisons such as ==, !=, <, and >= produce a Boolean result. Use and, or, and not to combine conditions. Use = to assign a value and == to compare values.

```
severity = "high"
if severity == "high":
    print("Review this alert")
```

Common mistake: using assignment where a comparison is required.

2. Conditional and iterative statements

An if statement chooses which block to run. elif checks another condition; else runs when none of the earlier conditions matched. Indentation defines the block.

```
status_code = 401

if status_code == 401:
    print("Authentication was rejected")
elif status_code == 200:
    print("Request succeeded")
else:
    print("Review the status code")
```

A for loop repeats an action for each item in a sequence. A while loop repeats while a condition remains true. Prefer a for loop when the input collection is already known; a while loop needs an update or exit condition to avoid running forever.

```
event_types = ["login_failure", "file_read", "login_failure"]

for event in event_types:
    if event == "login_failure":
        print("Flag for review")
```

Expected output is “Flag for review” twice. The script flags events for a person to inspect; it does not decide that an account is compromised.

3. Functions and readable code

A **function** groups a named task that can be called more than once. Parameters pass data into the function; return sends a result back. Functions make the purpose and boundaries of a script easier to review.

```
def is_failed_login(event_name):
    """Return True when an event label describes a failed login."""
    return event_name == "login_failure"

event = "login_failure"
if is_failed_login(event):
    print("Include in the review list")
```

Prefer clear names such as failed_login_count over unexplained abbreviations. Keep one function focused on one task. Add a short docstring when the purpose or expected input is not obvious. Separate reading input, processing it, and writing output so each part can be checked independently.

Libraries and modules

A **module** is a Python file or packaged collection of code that can be imported. The standard library includes modules for common tasks. For example, re provides regular-expression matching. Import only what the program needs and consult the module documentation rather than guessing function behavior.

```
import re

pattern = re.compile(r"\bfailed\b", re.IGNORECASE)
message = "Failed login for account"
if pattern.search(message):
    print("Matching event text")
```

4. Strings and lists

A **string** stores text. Strings support indexing, slicing, and methods such as strip(), lower(), split(), and replace(). These methods return a string; they do not silently change the original string.

A **list** stores an ordered collection. Lists can be indexed, looped over, appended to, or filtered into a new list.

```
raw_usernames = ["  ALI ", "Mina", "ali"]
usernames = [name.strip().lower() for name in raw_usernames]
print(usernames)
```

Example output:

```
['ali', 'mina', 'ali']
```

This normalizes spaces and letter case, but does not remove duplicate entries. Choose normalization carefully: changing case can be appropriate for some identifiers, but not every system treats names as case-insensitive.

A small defensive filtering example

The script below compares a fictional allow-list with a local list of addresses to remove. It returns a new list so the original list remains available for review.

```
allow_list = ["192.0.2.10", "192.0.2.25", "198.51.100.8"]
remove_list = ["192.0.2.25"]

remaining = [address for address in allow_list if address not in remove_list]
print(remaining)
```

Example output:

```
['192.0.2.10', '198.51.100.8']
```

The addresses use documentation-only ranges. In a real workflow, verify the source and authority of each change before updating a production allow-list.

5. Regular expressions

A **regular expression** (regex) describes a text pattern. Python's re module can search for, split, or replace text matching that pattern. A raw string such as r"\d+" helps keep backslashes in the regex readable.

Pattern	Meaning
\d	A digit
\d+	One or more digits
\w+	One or more word characters
^	Start of a string or line (depending on flags)
$	End of a string or line (depending on flags)
.	Any character except newline by default

Example: find event IDs in text where IDs use the literal form event- followed by digits.

```
import re

message = "review event-204 before closing the case"
match = re.search(r"\bevent-(\d+)\b", message)

if match:
    print(match.group(1))
```

Example output:

```
204
```

Regex matches are not proof that a string is valid, malicious, or safe. A pattern may match only part of a value. For structured formats such as JSON or CSV, use a parser designed for that format instead of a complex regex.

6. Automation and file handling

Automation combines repeatable steps so a person does not need to perform each one manually. Start with a read-only task and a small sample. Keep the input unchanged, write results to a separate output file, and inspect the result before relying on it.

The with open(...) pattern closes a file automatically. Set an encoding for predictable text handling. The mode "r" reads an existing file. The mode "w" can replace an existing file, so use it only for a deliberate output path. Do not point a learning script at sensitive or production data.

```
from pathlib import Path

input_path = Path("sample-events.txt")

with input_path.open("r", encoding="utf-8") as event_file:
    for line_number, line in enumerate(event_file, start=1):
        if "login_failure" in line:
            print(f"Line {line_number}: {line.strip()}")
```

Create a small local file named sample-events.txt in the same folder, with one fictional event per line. The program prints matching lines with their line numbers. It does not alter the file.

A safe automation workflow

1. State the task and allowed input clearly.
2. Preserve the original file and use a small test copy.
3. Parse the data with an appropriate library or simple line handling.
4. Validate assumptions, such as expected columns or event labels.
5. Print a summary or write to a new output file.
6. Review the output manually before taking any follow-up action.

7. Debugging and common mistakes

Debugging is the process of finding and fixing an error. Read a traceback from the bottom upward: the final line usually names the exception; preceding lines show where it occurred. Check line numbers, spelling, data types, indentation, file paths, and whether a variable contains the value you expect.

Problem	Likely cause	First check
SyntaxError	Invalid Python syntax	Read the indicated line and the line just before it

Problem	Likely cause	First check
NameError	A name was misspelled or not defined	Compare spelling and scope
TypeError	An operation received an unexpected type	Inspect values with type(value)
FileNotFoundError	Input path is wrong or file is absent	Check the working directory and filename
Unexpected empty match	Pattern does not match input as assumed	Print a sanitized sample and simplify the pattern
Loop never ends	The loop condition is never made false	Check the condition and update step

Do not “fix” a parsing error by blindly changing the input. Preserve a sample and identify the mismatch. Avoid printing passwords, tokens, personal data, or full sensitive log records while debugging.

Security and quality reminders

- Treat logs, filenames, and records as untrusted input; they may be malformed or intentionally crafted.
- Do not execute text read from a file as Python code.
- Do not hard-code passwords, API keys, or access tokens in scripts.
- Avoid destructive file modes or broad file operations in a learning exercise.
- Use clear variable names, small functions, comments for non-obvious decisions, and a reviewable output.
- Test only with data and systems you own or are explicitly authorized to use.
- Automation can repeat a mistake at scale; require human review before making access, block-list, or production changes.

Revision questions

1. What is the difference between a string and a list?
2. When would you choose if, for, or while?
3. What does a function's return statement do?
4. Why can a regex match be useful without proving that the whole value is valid?
5. Why is with open(..., encoding="utf-8") a useful pattern?
6. What risk does opening a file with mode "w" introduce?
7. How would you debug a FileNotFoundError without changing the input data?
8. Name two checks you would perform before trusting an automated security report.

References

- Google video: Fundamentals of Python for Cybersecurity
- Google Cybersecurity Certificate curriculum
- Python tutorial: control flow, data structures, modules, and files
- Python tutorial: reading and writing files
- Python regular expression HOWTO
- Python re module reference